

Calcolo Numerico 2025-26

Homework 1 – Zeri di funzione

Consegna

Zeri di funzione: utilizzare il metodo di bisezione, il metodo delle iterazioni di punto fisso e il metodo di Newton per il calcolo dello zero delle seguenti funzioni. Disegnare il grafico della funzione per localizzare lo zero e scegliere sia l'intervallo in cui applicare la bisezione che il punto iniziale per gli altri due algoritmi.

1. $f(x) = \ln(x+1) - x$. ($g(x) = \ln(x+1)$)
2. $f(x) = x^2 - \cos(x)$. ($g(x) = \sqrt{\cos(x)}$)
3. $f(x) = \sin(x) - x^2$. ($g(x) = 2\sin(x)$)
4. $f(x) = e^x - 3x$. ($g(x) = \frac{1}{3}e^x$)

Codici degli algoritmi

Metodo di bisezione

```
1 def metodo_bisezione(f, a, b, max_iter, epsilon):
2     cond = True
3     i = 0
4     if f(a) * f(b) >= 0.0:
5         return None, 0
6     while cond and i < max_iter:
7         c = (a + b) / 2
8         if abs(f(c)) < epsilon:
9             cond = False
10        if cond and f(a) * f(c) < 0:
11            b = c
12        elif cond:
13            a = c
14            i = i + 1
15    return c, i
```

Metodo delle iterazioni di punto fisso

```
1 def metodo_punto_fisso(g, x0, max_iter, epsilon1, epsilon2):
2     i = 0
3     cond = True
```

```

4     xk = x0
5     while cond and i < max_iter:
6         x = g(xk)
7         if abs(x - xk) < epsilon1 or abs(f(x)) < epsilon2:
8             cond = False
9         xk = x
10        i = i + 1
11    return xk, i

```

Metodo di Newton

```

1     def metodo_newton(f, df, x0, max_iter, epsilon1, epsilon2):
2         i = 0
3         cond = True
4         xk = x0
5         if abs(df(xk)) < 1e-10:
6             return None, 0
7         while cond and i < max_iter:
8             xk1 = xk - f(xk) / df(xk)
9             if abs(xk1 - xk) < epsilon1 or abs(f(xk1)) < epsilon2:
10                cond = False
11            xk = xk1
12            i = i + 1
13    return xk, i

```

Risultati delle esecuzioni

Prima funzione: $f(x) = x^2 - \cos(x)$. ($g(x) = \sqrt{\cos(x)}$)

Metodo	Punto iniziale	Intervallo	Epsilon	Soluzione	Iterazioni
Bisezione	0	[0,1]	10^{-6}	0.824131965637207	20
Punto Fisso	0	[0,1]	10^{-6}	0.824132109264407	18
Newton	0	[0,1]	10^{-6}		

Metodo	Punto iniziale	Intervallo	Epsilon	Soluzione	Iterazioni
Bisezione	0.5	[0,1]	10^{-6}	0.824131965637207	20
Punto Fisso	0.5	[0,1]	10^{-6}	0.824132596440194	17
Newton	0.5	[0,1]	10^{-6}	0.824132312409912	4

Metodo	Punto iniziale	Intervallo	Epsilon	Soluzione	Iterazioni
Bisezione	0.8	[0,1]	10^{-6}	0.824131965637207	20
Punto Fisso	0.8	[0,1]	10^{-6}	0.824132025062060	14
Newton	0.8	[0,1]	10^{-6}	0.824132376563292	2

Metodo	Punto iniziale	Intervallo	Epsilon	Soluzione	Iterazioni
Bisezione	0.8	[0,1]	10^{-5}	0.824134826660156	17
Punto Fisso	0.8	[0,1]	10^{-5}	0.824125006492905	10
Newton	0.8	[0,1]	10^{-5}	0.824132376563292	2

Metodo	Punto iniziale	Intervallo	Epsilon	Soluzione	Iterazioni
Bisezione	0.8	[0,1]	10^{-4}	0.8240966796875	13
Punto Fisso	0.8	[0,1]	10^{-4}	0.824215052092387	8
Newton	0.8	[0,1]	10^{-4}	0.824132376563292	2

Metodo	Punto iniziale	Intervallo	Epsilon	Soluzione	Iterazioni
Bisezione	0.8	[0,1]	10^{-3}	0.82421875	8
Punto Fisso	0.8	[0,1]	10^{-3}	0.824549519096654	5
Newton	0.8	[0,1]	10^{-3}	0.824470434030340	1

Metodo di Newton

```

1  def metodo_newton(f, df, x0, max_iter, epsilon1, epsilon2):
2      i = 0
3      cond = True
4      xk = x0
5      if abs(df(xk)) < 1e-10:
6          return None, 0
7      while cond and i < max_iter:
8          xk1 = xk - f(xk) / df(xk)
9          if abs(xk1 - xk) < epsilon1 or abs(f(xk1)) < epsilon2:
10             cond = False
11             if cond:
12                 xk = xk1
13                 i = i + 1
14     return xk, i

```